

Отчёт по выполнению лабораторной работы №1

Работа с группами

Коровкин Н. М.

9 сентября 2026

Российский университет дружбы народов, Москва, Россия

Информация

- Коровкин Никита Михайлович
- Студент
- Российский университет дружбы народов
- 1132246835@pfur.ru

Выполнение лабораторной работы

1. Установка необходимого программного обеспечения

Для выполнения лабораторной работы на компьютере с macOS были установлены UTM, Packer и Vagrant. Для установки Packer и дополнительных инструментов использовался Homebrew.

Для установки UTM и Packer были выполнены команды:

`brew install --cask utm`
`brew install packer`
`wget Vagrant` устанавливался с использованием официального установщика. После установки Vagrant был подключён провайдер `vagrant_utm`:

`vagrant plugin install vagrant_utm`
После установки была выполнена проверка наличия плагина:

`vagrant plugin list | grep vagrant_utm`
Провайдер `vagrant_utm` позволяет Vagrant управлять виртуальными машинами непосредственно через UTM. В отличие

Выполнение лабораторной работы

2. Подготовка рабочего каталога и образа Rocky Linux

Для лабораторной работы была создана структура каталогов проекта:

```
mkdir -p ~/labs/user_name/packer/http mkdir -p
```

```
~/labs/user_name/vagrant/provision/default mkdir -p
```

```
~/labs/user_name/vagrant/provision/server mkdir -p
```

```
~/labs/user_name/vagrant/provision/client
```

После этого был выполнен переход в каталог Packer:

```
cd ~/labs/user_name/packer
```

Для виртуальных машин Apple Silicon необходимо использовать образ Rocky Linux архитектуры AArch64 (ARM64). Поэтому был загружен минимальный ISO-образ Rocky Linux 10.1 для AArch64, а также файл CHECKSUM:

```
wget https://dl.rockylinux.org/pub/rocky/10/isos/aarch64/Rocky-10.1-aarch64-
```

3. Подготовка Kickstart-файла

Для автоматической установки Rocky Linux был подготовлен Kickstart-файл:

`http/rocky10-ks.cfg` Данный файл используется Racker при установке операционной системы и позволяет автоматически выполнить необходимые действия во время установки.

Для ARM64 был скорректирован параметр загрузчика. В используемой конфигурации применялся вариант:

`bootloader -- append="console=ttyAMA0,115200n8 net.ifnames=0" --timeout=1`

Также в параметрах загрузки использовалась передача Kickstart-файла:

`inst.ks=http://{ .HTTPIP }:{ .HTTPPort }/rocky10-ks.cfg` В данном варианте отдельное использование cloud-init не требуется. Настройка статического

IP адреса производится позднее при помощи `nmcli` в `provision` скрипте (рис

Выполнение лабораторной работы

4. Сборка Vagrant Box при помощи Packer

Для создания собственного Box был подготовлен конфигурационный файл Packer.

В конфигурации указываются ISO-образ Rocky Linux, его контрольная сумма, каталог с Kickstart-файлом, имя пользователя и пароль, а также команды загрузки и завершения работы виртуальной машины.

Основные параметры имеют следующий вид:

```
iso_url = "/полный/путь/к/Rocky-10.1-aarch64-minimal.iso" iso_checksum =  
"sha256:ХЭШ_ИЗ_CHECKSUM" http_directory = "/полный/путь/к/packer/http"  
ssh_username = "vagrant" ssh_password = "vagrant" shutdown_command = "echo  
'vagrant' | sudo -S /sbin/halt -h -p" Для загрузки Kickstart-файла используется:
```

```
boot_command = [ "o" " inst ks=http://[[ HTTP_IP ]]:[[ HTTP_PORT ]]/rocky10 ks.cfs
```

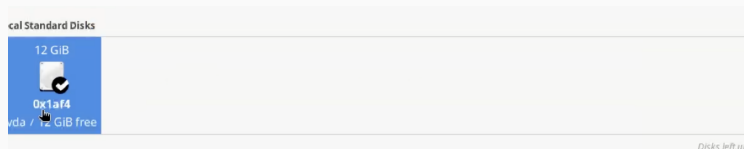
5. Регистрация Box в Vagrant

После завершения сборки был получен файл:

rockylinux10-utm.box Созданный Box был зарегистрирован в Vagrant следующей командой:

`vagrant box add rockylinux10 rockylinux10-utm.box` Для проверки регистрации был выполнен запрос:

`vagrant box list` В результате в списке доступных Box появился rockylinux10.(рис. (fig:005?)).



Создание и настройка Vagrantfile

Для управления виртуальными машинами был создан файл Vagrantfile.

В качестве базового образа используется созданный ранее Box:

`config.vm.box = "rockylinux10"` В проекте определены две виртуальные машины:

`server` `client` Для каждой машины задаётся имя хоста, количество процессоров и объём оперативной памяти.

Для сервера:

`server.vm.hostname = "server"`

`server.vm.provider "utm" do |u| u.name = "server" u.cpus = 2 u.memory = 2048 end`

Для клиента аналогично задаются параметры:

`client.vm.hostname = "client"`

Выполнение лабораторной работы

7. Первый запуск виртуальной машины

После подготовки конфигурации был выполнен переход в каталог Vagrant-проекта:

`cd ~/labs/user_name/vagrant` Затем была запущена виртуальная машина сервера:

`vagrant up server` При первом запуске автоматически открывается приложение UTM, в котором появляется созданная виртуальная машина `server`.

Для подключения к виртуальной машине по SSH используется:

`vagrant ssh server` После проверки подключения был выполнен выход из виртуальной машины:

`exit` Для остановки сервера используется:

8. Настройка приватной сети между server и client

Для организации отдельной сети между виртуальными машинами был добавлен дополнительный сетевой адаптер.

Провайдер `vagrant_utm` не позволяет задать `private_network` непосредственно в `Vagrantfile`, поэтому третий сетевой адаптер необходимо добавить вручную через графический интерфейс UTM.

Сначала виртуальная машина была остановлена:

`vagrant halt server` Затем в UTM для виртуальной машины `server` был открыт раздел:

Edit ☒ Network ☒ New Для нового адаптера был выбран режим:

Host Only Такая же настройка была выполнена для виртуальной машины `client`.

9. Назначение статического IP-адреса и проверка соединения

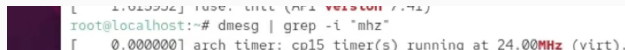
Для сервера был подготовлен дополнительный provision-скрипт:

provision/server/03-static-ip.sh В скрипте используется утилита nmcli

Имя сетевого соединения необходимо предварительно проверить внутри виртуальной машины командой:

nmcli connection show После настройки сервера была выполнена проверка связи с клиентской виртуальной машины:

vagrant ssh client ping -c3 192.168.1.1 Успешное получение ответов на ICMP-запросы подтверждает наличие сетевого соединения между клиентом и сервером.(рис. (fig:009?)).



```
root@localhost:~# nmcli connection show
[ 0.000000] arch timer: cp15 timer(s) running at 24.00MHz (virt).
```

Ответ на контрольные вопросы

1. Что такое Vagrant?

vagrant — инструмент для создания и управления виртуальными машинами при помощи конфигурационных файлов и командной строки.

2. Для чего используется Packer?

Packer используется для автоматической сборки образов виртуальных машин. В данной лабораторной работе он применяется для создания Vagrant Box из ISO-образа Rocky Linux.

3. Что такое Vagrant Box?

Vagrant Box — это упакованный образ виртуальной машины, который может использоваться Vagrant как исходный образ для создания новых виртуальных машин.

В ходе лабораторной работы были получены практические навыки использования UTM, Packer и Vagrant для создания и управления виртуальными машинами.

